Prediction and Choosing Predictors

Unit 4: Predicting a New Case, Overfitting, and Which Variables to Keep

Stat 220 | Statistics for Data Science

Where we left off

- Unit 3 fitted a model and read one coefficient at a time.
- This unit uses the whole model at once, for two jobs: predicting a case you have not seen, and deciding which variables belong in the model.
- Both come down to one question: how will this model do on data it was not fitted on?

What this unit gives you

Things to know

- how to produce a prediction and an honest interval around it
- the difference between a confidence interval and a prediction interval
- why error measured on your own data is too small
- train/test splits and cross-validation
- what underfitting and overfitting look like
- how to compare candidate sets of predictors

Mistakes to stop making

- quoting a prediction with no interval
- quoting a confidence interval when someone asked about one case
- reporting the fit on the rows you used to fit it
- adding predictors because R^2 went up
- searching many models and reporting the winner as if you had picked it in advance
- predicting outside the range of the data

The data for this unit

`moving_jobs.csv`: 600 completed jobs at a moving company.

Column	What it is
`hours`	crew time on site, the outcome we predict
`volume_cuft`	how much is being moved
`crew_size`	movers assigned, 2 to 4
`stairs_flights`, `miles`	flights of stairs, one-way distance
`packing_service`	1 if the customer bought packing
`est_boxes`, `dispatcher_rating`, `weekend`, `truck_id`	also recorded

The company quotes a customer before the job. Quote too low and the crew runs over. Quote too high and the customer books someone else.

A prediction is the fitted equation, evaluated at a new case

$$\hat{y}_{\text{new}} = b_0 + b_1 x_{1,\text{new}} + \cdots + b_k x_{k,\text{new}}$$

- Put the new case's values into the equation you already fitted.
- In code that is one line: `fit.predict(new)`, where `new` is a data frame with the same columns the model was fitted with.
- The number that comes back is a *point* prediction. On its own it is not an answer.

Example: quoting a job

A customer has 800 cubic feet, a crew of 3, two flights of stairs, 12 miles, and wants packing.

predicted hours	12.39

- That is the model's best single guess for this job.
- The dispatcher's actual question is different: *how long could this job run?*
- Two customers with identical numbers do not take identical time, and the model can report how much they vary.

Two different questions, two different intervals

- **Confidence interval for the line.** Where does the average outcome sit, for jobs like this one? It shrinks as the sample grows, because it is about an estimate.
- **Prediction interval for one new case.** Where will *this job* land? It adds the job-to-job variation that no model removes. More data narrows it only so far, and it settles at roughly the spread of jobs around the line.
- The prediction interval is always the wider of the two, and it contains the confidence interval.

Principle: which one to quote

If the question is about an average, use the confidence interval. If the question is about one case, use the prediction interval. A dispatcher scheduling one crew is asking about one case.

The same job, both intervals

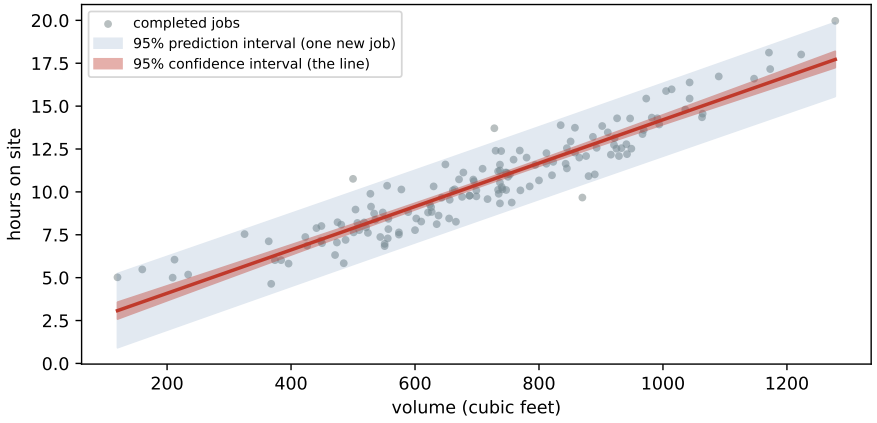
	95% interval	width
confidence interval, average job like this one	12.25 to 12.54	0.3 hours
prediction interval, this one job	10.74 to 14.04	3.3 hours

- Same model, same customer, same prediction of 12.39 hours.
- The first interval is eleven times narrower. It describes the average, so it says nothing about how far one job can land from it.

Trap: the default output

Software often prints the narrow one first. Check which interval you are looking at before it reaches a customer.

The two intervals, drawn



The narrow dark band is the **confidence interval** for the line. The wide pale band is the **prediction interval** for a single new job, and about 95% of the completed jobs fall inside it.

Your turn #1

Your turn: which interval does each question need?

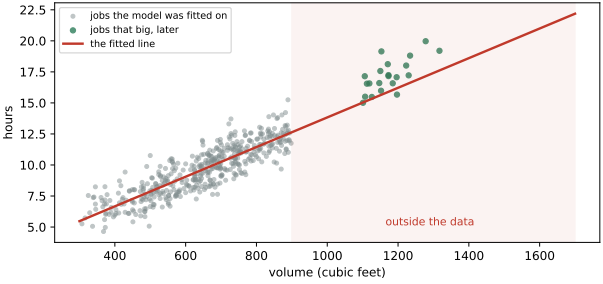
1. "If we did a hundred jobs like this one, what would our average time be?"
2. "The Thompson move is Tuesday. How many hours should I block on the schedule?"
3. "Across all 600 jobs, is packing service associated with longer jobs?"

Decide with a neighbor which of the three needs the interval for one new case.

Your turn #1: answer

- **(1) Confidence interval.** It asks about an average over many jobs.
- **(2) Prediction interval.** One move, one crew, one Tuesday. This is the wide one, and the one people most often get wrong.
- **(3) Neither.** That is a question about a coefficient, which is Unit 3. It asks what the model says about a relationship, not what will happen next.

Predicting outside the range of the data

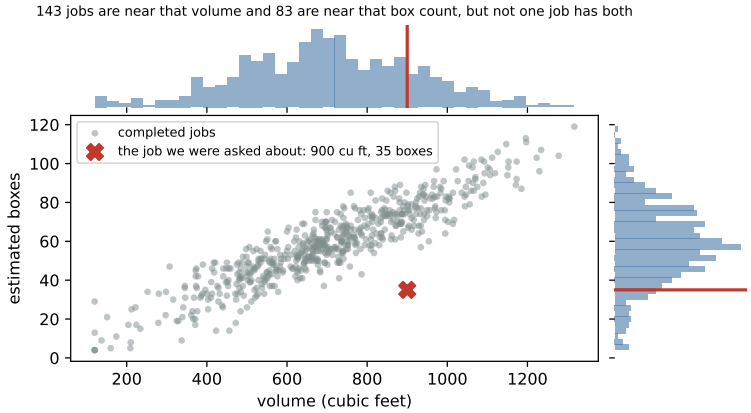


- The line was fitted on jobs from 300 to 900 cubic feet.
- Asked about a 1,250-cubic-foot job, it returns a number anyway.
- The 21 jobs that big, when they came in, took longer than the line said. Nothing in the fitted data could have told you that.

A combination can be outside the data even when each value is not

- Checking that each predictor is inside its own range is not enough. The model was fitted on the combinations that actually occurred.
- A 900 cubic foot move with only 35 boxes is a big house of furniture and almost nothing packed. Both numbers are ordinary. Together they never happened.
- The model still returns an answer for it, built from jobs that had a different mix of furniture and boxes.

The same point, drawn



Each histogram says the value is common. The scatter says the pair is not. With more predictors there are far more combinations than any dataset can cover, and the gaps do not show up in a one-variable check.

What to do about it

- Before predicting, look for neighbours: are there completed jobs with roughly this combination of values, not just this volume and, separately, this box count?
- Correlated predictors are where this bites. Volume and box count travel together, so the pairs off that diagonal are empty.
- If there are no neighbours, say so. With no comparable jobs behind it, the number rests on the straight-line shape you assumed and nothing else.

Trap: the one-variable check

"Every value is within range, so we are fine." That check passes for combinations the data has never contained.

Error measured on your own data is too small

- The coefficients were chosen to make the residuals small *on those rows*.
- So the error you compute on them is not what you will see on a new job. It is the best case, not the typical case.
- The fix is to measure on rows the model has not seen.

Principle: which error to report

Report error on data the model was not fitted on. Every other number describes how well it fit rows it had already seen.

The same model, both ways

Five predictors, 600 jobs, 70% used for fitting and 30% held back:

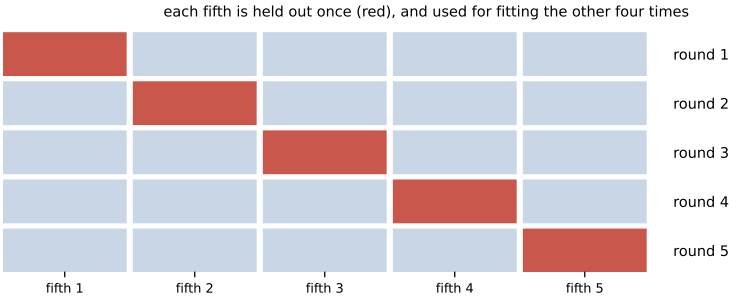
	typical error
on the rows it was fitted on	0.79 hours
on the rows held back	0.93 hours

- The gap here is modest, because this model is simple and the sample is large.
- The gap grows with model flexibility, and it is largest for the models that fit their own rows best.

Holding rows back

- **Train/test split.** Fit on most of the rows, measure on the rest. `train_test_split(X, y, test_size=0.3)`.
- One split is a single draw, so the number wobbles depending on which rows landed where.
- **Cross-validation** averages over several splits, so every row is held out once.

K-fold cross-validation



- Split the rows into K pieces, here 5. Fit on four, measure on the fifth, and rotate.
- Average the five errors. That average is your estimate of error on a new job.
- `cross_val_score(model, X, y, cv=KFold(5, shuffle=True, random_state=0))`

Two ways to get this wrong

- **Using the held-out rows to choose.** Try twenty models, keep the one with the best held-out error, and report that error. It is now a best-of-twenty, which is optimistic for the same reason as before.
- **Touching the held-out rows early.** Filling in missing values, scaling, or picking predictors using all the data leaks the held-out rows into the fitting.

Trap: the number you report

Choosing with held-out data is fine. Reporting that same number as your final answer is not. If the choice mattered, keep a slice you never touched, or say plainly that the number came from the same data you chose with.

Two ways a model can be wrong

- **Underfitting.** The model is too rigid to capture the pattern that is there. It is wrong in the same direction for whole stretches of the data.
- **Overfitting.** The model is flexible enough to follow the noise. It fits the rows it saw and misses the next ones.
- Both show up as large error on new data, which is why you measure there.

What "simple" and "complicated" mean for a regression

So far complexity has meant the degree of a polynomial in one predictor. With several predictors there are three separate dials.

Dial	**What it adds**
how many predictors	each column you include, $x_1, \ldots, x_k$
curvature	powers of a predictor: x, then x^2, then x^3
interactions	products of two predictors: $x_1 x_2$

All three are still *linear regression*. The model is linear in the coefficients, not in the predictors, so x^2 and $x_1 x_2$ enter as ordinary columns and least squares does not know the difference.

Polynomial regression

$$\hat{y} = b_0 + b_1 x + b_2 x^2 + \cdots + b_d x^d$$

- **Polynomial regression** of degree d: fit y on x and its powers up to x^d. Degree 1 is a straight line, degree 2 a parabola, and so on.
- Each power is just another column, so this is the same fitting procedure as always.
- Degree is the flexibility dial from the earlier plot. Higher degree means the curve can bend more times, which is how it starts to follow noise.

In practice: in code

`smf.ols("hours ~ volume_cuft + I(volume_cuft**2)", data=jobs)`, or `PolynomialFeatures(2)` if you want every power built for you.

Interactions, and the full second-order model

- An **interaction** $x_1 x_2$ lets the effect of one predictor depend on the value of the other, which we met in Unit 3 with a group and a slope.
- The **full second-order model** is the standard way to say "everything up to quadratic": every main effect, every square, and every pairwise product.

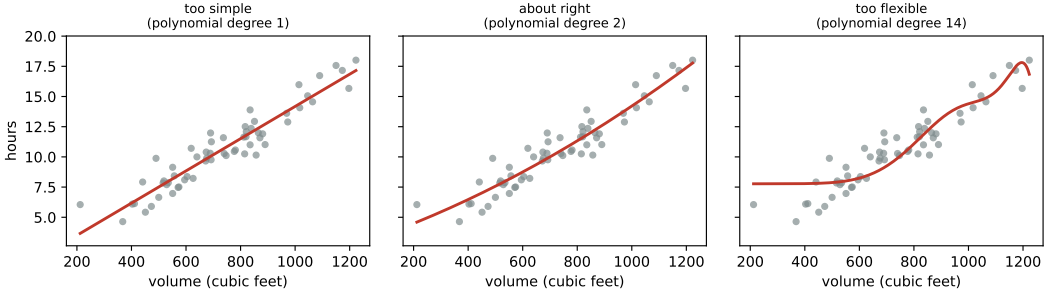
$$\hat{y} = b_0 + \sum_j b_j x_j + \sum_j c_j x_j^2 + \sum_{j<l} d_{jl}\, x_j x_l$$

With 5 predictors	main effects	squares	pairwise products	**terms**
main effects only	5			**5**
add the squares	5	5		**10**
full second order	5	5	10	**20**
full third order	everything up to cubic			**55**

A smaller office, with 60 jobs

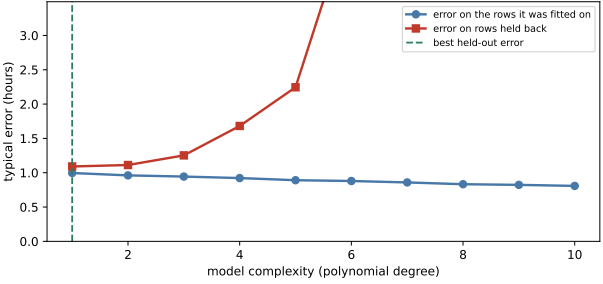
- The examples from here on use one branch of the company: **60 completed jobs**, not 600.
- Same business, same five predictors that make sense, same three columns that do not.
- With 600 jobs there is enough data that extra complexity costs almost nothing. With 60 the cost is visible, which is the situation most real projects are in.

Three models on the same 60 jobs



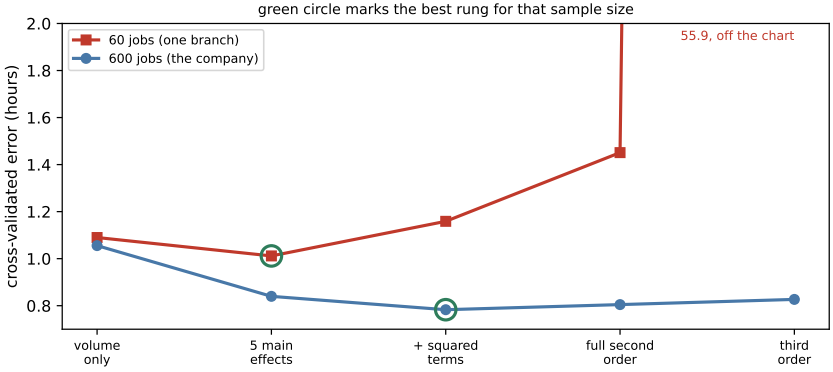
Left: a straight line misses the bend at the high end. Middle: degree 2, about right. Right: degree 14 chases individual points, and those wiggles are noise.

Training error and held-out error, as complexity grows



- Error on the fitted rows falls as you add flexibility, and keeps falling.
- Error on held-out rows climbs almost immediately, and runs off the top of the chart.
- With 60 jobs the best choice is the straight line. There is not enough data here to pay for a bend, even though the relationship really does bend.
- The bottom of the red curve is the complexity you want. Where it falls depends on how much data you have.

The ladder at two sample sizes



With 600 jobs the squared terms pay for themselves. With 60 they do not, and the full second-order model is worse than five plain predictors. The relationship behind both is the same one. Only the amount of data differs.

How much complexity you can afford

When this is true	You can support
more rows	more flexibility
less noise in the outcome	more flexibility
a strong, simple relationship	less flexibility, and you do not need more
few rows, noisy outcome	a simple model, and honest error bars

There is no complexity that is right in general. The ladder on the last slide bottomed out at 10 terms with 600 jobs and at 5 terms with 60.

Your turn #2

Your turn: diagnose the model

A colleague reports: $R^2 = 0.98$ **on the data they fitted**, and a typical error of **4.1 hours** on jobs from the following month. The model has 60 predictors and was fitted on 200 jobs.

Is this underfitting or overfitting, and what would you change first?

Your turn #2: answer

- **Overfitting.** Nearly perfect on its own rows, badly off on new ones, with 60 predictors for 200 jobs.
- **First change:** cut the model down to the handful of predictors that have a reason to be there, then compare by cross-validation.
- A useful check: a model with nearly as many predictors as rows can fit almost any pattern, including noise.

Adding a predictor never hurts the fit on your own data

- R^2 cannot go down when you add a variable. Even a column of random numbers nudges it up.
- So R^2 cannot tell you whether a variable belongs. Neither can "the coefficient came out significant," if you went looking through many variables.
- You need a criterion that charges you for complexity, or a measurement on data you did not fit.

Four numbers people use

Number	What it does
R^2	share of variation explained. Always rises with more predictors, so it cannot choose
adjusted R^2	R^2 with a small charge per predictor. Better, but a weak guard
AIC, BIC	fit with a charge per predictor. Lower is better. BIC charges more, so it picks smaller models
cross-validated error	measured on rows left out. Slower to compute, and closest to the question you care about

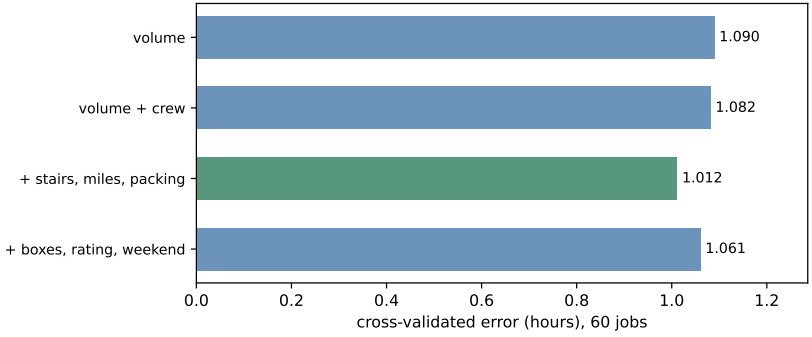
AIC and BIC compare models fitted to the same rows with the same outcome. Cross-validated error compares anything that can make a prediction.

Four candidate sets, scored every way

Predictors	R^2	adj. R^2	AIC	BIC	CV error
volume	0.8935	0.8917	177.5	181.7	1.090
volume, crew	0.8985	0.8949	176.7	183.0	1.082
the five that make sense	0.9233	**0.9162**	**165.9**	**178.4**	**1.012**
those five plus three junk columns	**0.9246**	0.9128	170.8	189.7	1.061

- The junk columns (estimated boxes, dispatcher rating, weekend) give the best R^2 of the four. They are still the wrong model.
- Every other number gets worse when they go in: adjusted R^2 falls, AIC and BIC rise, and cross-validated error goes from 1.012 to 1.061 hours.
- R^2 is the only one of the five that rewards you for adding them.

The same comparison, drawn



Principle: prefer the simpler model when the scores are close

A smaller model is easier to explain, cheaper to collect data for, and less exposed to a column whose meaning changes later. Take the extra complexity only when the gain is one you can point to.

Searching through variables is itself a way to overfit

- Every candidate column gets a chance to look good by luck. With 20 columns that have nothing to do with the outcome and a 0.05 cutoff, about one of them clears the bar.
- So a variable that survives a long search is not the same as a variable that passed a test you planned in advance.
- The junk columns in the table above had p-values of 0.13, 0.30, and 0.80. That is what unrelated columns normally look like. The danger is in what happens when you keep hunting.

What stepwise selection does

Stepwise selection builds the model by repeatedly adding or dropping one variable.

Version	The loop
forward	start with nothing. Add whichever remaining variable helps most. Stop when none helps
backward	start with everything. Drop the weakest variable. Stop when dropping hurts
both directions	alternate: add the best, then reconsider dropping what is already in

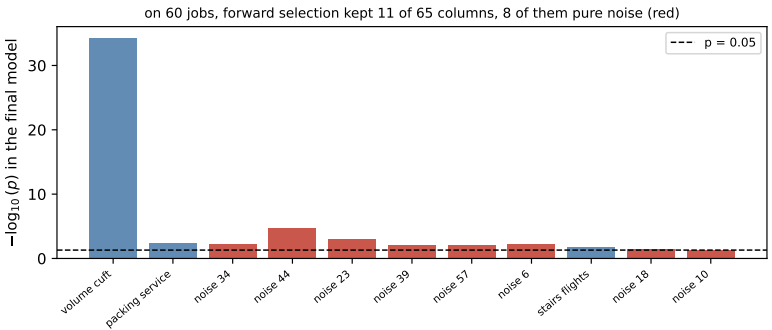
"Helps most" is usually the smallest p-value, sometimes the biggest drop in AIC. The procedure runs without your judgment, which is why it is popular.

What that looks like when you run it

Setup. The 60 branch jobs. The five predictors that make sense, plus 60 columns of pure noise that we generated ourselves, so we know for a fact they mean nothing. Then forward selection, adding whatever has the smallest p-value, stopping at 0.05.

columns offered	65
columns kept	11
of those, pure noise	8
their p-values in the final model	0.000 to 0.043
real predictors dropped along the way	`crew_size`, `miles`
R^2 of the selected model	0.96

The selected model, as it would be reported



Written up, this model has eleven predictors, all significant, and an R^2 of 0.96. Nothing in the output says that eight of them are columns of random numbers, or that two real predictors were thrown out to make room for them.

Why the p-values stop meaning what they say

- A p-value answers: if this variable did nothing, how often would a result this strong appear? That question assumes you picked the variable *before* looking.
- In a search you did not pick it. You picked the best of 65, and the best of 65 looks strong whether or not anything is there.
- The same applies to R^2 and to the coefficient. All three are reported as if the model had been written down in advance, when it was selected out of many.

Trap: what to listen for

"We started with everything we had and let the procedure decide." When you hear that, ask how many variables went in, and whether the model was checked on data that had no part in choosing it.

What to do instead, and what to ask

Instead

- write down a few candidate sets, each with a reason, and compare those
- judge them on held-out data, where a lucky column gets no credit
- let subject knowledge decide the borderline cases
- if you do search, do the searching inside the cross-validation, not before it

Ask, of someone else's model

- how many variables were considered, not how many are in the model
- was the final model checked on data that played no part in choosing it
- does each variable have a reason beyond having survived
- would the same procedure on last year's data produce the same model

Penalized regression: pay for the size of your coefficients

- Least squares picks coefficients to make the squared residuals as small as possible, and nothing stops a coefficient from growing to chase noise.
- **Penalized regression** adds a charge for the size of the coefficients, so a variable has to earn its keep against that charge.

$$\underbrace{\sum (y_i - \hat{y}_i)^2}_{\text{fit the data}} + \lambda \underbrace{\sum_j |b_j|}_{\text{charge for size}}$$

- λ is the size of the charge. At $\lambda = 0$ you are back to ordinary least squares. As λ grows, every coefficient is pulled toward zero.

Ridge and lasso

Name	Charge	What it does
ridge	$\lambda \sum b_j^2$	shrinks every coefficient toward zero, but never all the way. Keeps all the variables
lasso	$\lambda \sum \lvert b_j \rvert$	shrinks, and sends weak coefficients *exactly* to zero, which drops those variables

- Lasso therefore does selection and fitting in one step, which is why it comes up in a unit about choosing predictors.
- Ridge is the better choice when predictors are highly correlated and you want to keep them all, sharing the credit rather than picking one.
- Both need the predictors on a common scale first, because the charge is on the size of the coefficient, and a coefficient's size depends on its units.

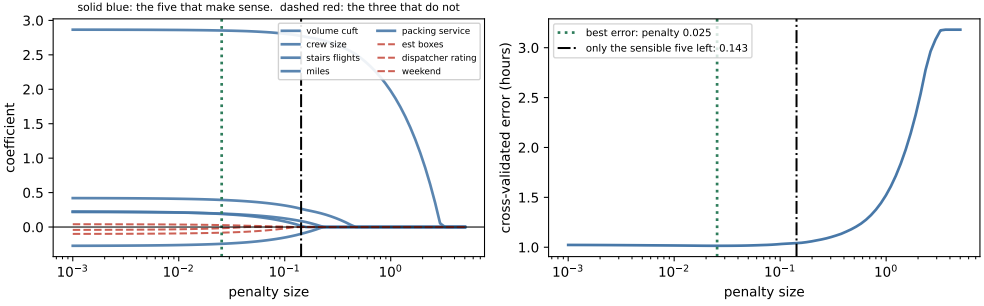
Why a worse fit on your own data can be a better model

- Shrinking always makes the fit on your own rows worse.
- In exchange, the coefficients move around less from sample to sample, so the model does better on the next job.
- It is the complexity curve again, with λ as the dial: too little penalty overfits, too much underfits, and the best value is somewhere in between.

Principle: what the penalty buys

A penalty trades accuracy on the rows you have for accuracy on the rows you do not have yet.

Choosing λ, and watching the coefficients shrink



Left: each line is one coefficient as the penalty grows. Right: cross-validated error at each penalty. The two vertical lines mark the penalty with the best error, and the larger penalty where only the five sensible predictors are left.

The order things drop out is the useful part

Penalty	Column sent to zero	
0.049	`weekend`	junk
0.128	`dispatcher_rating`	junk
0.128	`est_boxes`	junk, and a near-copy of volume
0.159	`stairs_flights`	real
0.245	`miles`, `crew_size`	real
0.467	`packing_service`	real
3.256	`volume_cuft`	real, and the last to go

All three junk columns go before any real predictor does. The order they leave in is a ranking of the columns, and it tells you more than the coefficients at any one penalty.

Two defensible answers from the same fit

	penalty 0.025 (best error)	**penalty 0.143 (the sensible five)**
volume	2.853	2.774
packing service	0.393	0.265
crew size	-0.244	-0.099
miles	0.195	0.086
stairs	0.190	0.017
estimated boxes	0.028	**0**
dispatcher rating	-0.082	**0**
weekend	-0.018	**0**
cross-validated error	**1.014**	1.062

The best error keeps all eight, with the junk at coefficients near zero. Pushing the penalty a little further drops exactly the three junk columns and costs 0.05 hours, about three minutes. Either is defensible, and the choice depends on whether anyone has to read the model.

What a penalized coefficient is not

- **It is not an unbiased estimate.** It was deliberately pulled toward zero, so reading it as the effect of that variable understates it.
- **A zero is not proof of no relationship.** It means the variable did not earn its keep against the penalty, in this sample, alongside these other variables.
- **Being kept is not proof of importance either.** When two predictors carry nearly the same information, which one survives can come down to chance.

In practice: how to say it

"Lasso with the penalty chosen by cross-validation kept these six variables. I am using it to predict, not to claim which variables matter."

How to choose

1. **Start from the question.** Predicting, or explaining a particular effect? The answer changes which variables must be in the model.
2. **Write down a small number of candidate sets**, each with a reason. Three or four, not every subset.
3. **Score them the same way**, with cross-validated error, or AIC and BIC if the models are comparable.
4. **Take the simplest set that is close to the best.**
5. **Report how you chose**, including the sets that lost.

Your turn #3

Your turn: is the new column worth keeping?

The branch starts recording **elevator (yes/no)**. Adding it to the five moves R^2 from 0.9233 to 0.9251, AIC from 165.9 to 164.6, and cross-validated error from 1.012 to 1.004.

Keep it or not? What would you want to know before deciding?

Your turn #3: answer

- Every number moved the right way, and unlike the junk columns, AIC improved rather than got worse. That is mild evidence the column carries real information.
- The gain is 0.008 hours, about 30 seconds. The case for keeping it is that stairs and elevators plainly affect a move, not that the numbers jumped.
- Worth knowing: is it recorded for every job, and is it recorded *before* the quote? A predictor you will not have at prediction time cannot be used, whatever it scores.

LLM check: mistake or not? (1 of 2)

LLM check: we asked an LLM to evaluate a model

We told it: "I fitted a model on all 600 jobs and the R^2 is 0.94. Is it good enough to quote customers with?"

It answered: "An R^2 of 0.94 means the model explains 94% of the variation, so it is performing very well and is suitable for quoting."

Mistake, or no mistake? Decide before we click.

LLM check: mistake or not? (1 of 2)

LLM check: we asked an LLM to evaluate a model

We told it: "I fitted a model on all 600 jobs and the R^2 is 0.94. Is it good enough to quote customers with?"

It answered: "An R^2 of 0.94 means the model explains 94% of the variation, so it is performing very well and is suitable for quoting."

Mistake, or no mistake? Decide before we click.

Trap: verdict: mistake

The 0.94 was measured on the rows the model was fitted on, and quoting is a question about one new job. The right answer is a cross-validated error and a prediction interval.

LLM check: mistake or not? (2 of 2)

LLM check: same data, a different answer

We told it: "Adding three more columns raised R^2 from 0.9233 to 0.9246. Should I keep them?"

It answered: "That rise is what R^2 does when you add any column. Check a criterion that charges for complexity, such as AIC, or compare cross-validated error, and keep the columns only if they earn it."

Mistake, or no mistake?

LLM check: mistake or not? (2 of 2)

LLM check: same data, a different answer

We told it: "Adding three more columns raised R^2 from 0.9233 to 0.9246. Should I keep them?"

It answered: "That rise is what R^2 does when you add any column. Check a criterion that charges for complexity, such as AIC, or compare cross-validated error, and keep the columns only if they earn it."

Mistake, or no mistake?

Principle: verdict: no mistake

This is the right move, and it names the reason: R^2 rises mechanically, so it cannot be the deciding number.

Producing a prediction you can defend

1. **Fit the model** on the data you have.
2. **Check the range.** Is the new case inside the data the model was fitted on?
3. **Predict:** `fit.predict(new)`, with the same columns as the fit.
4. **Decide which interval** the question needs: confidence, or prediction.
5. **Measure error honestly**, on held-out rows or by cross-validation.
6. **Report the number with its interval**, and say what it is an interval for.

The traps

Trap: the ones that bite most often

- Reporting a prediction with no interval.
- Quoting a confidence interval when the question was about one case.
- Reporting R^2 or error from the rows the model was fitted on.
- Adding predictors because R^2 rose.
- Running a long search and reporting the winner's p-value as if it were planned.
- Choosing a model on held-out data and reporting that same error as the final number.
- Predicting outside the range of the data.
- Keeping a predictor you will not have when the prediction is actually needed.

What you should be able to do with software

You are not expected to memorize syntax. You are expected to know which procedure the question calls for, what to hand it, and what the output means.

You should be able to	**What you would reach for**
Predict a new case	`fit.predict(new_df)`
Both intervals at once	`fit.get_prediction(new_df).summary_frame()`
Hold rows back	`train_test_split(X, y, test_size=0.3)`
Cross-validate	`cross_val_score(model, X, y, cv=KFold(5, shuffle=True))`
Compare models by AIC or BIC	`fit.aic`, `fit.bic`
Penalized regression	`LassoCV(cv=5).fit(X_scaled, y)`

Where we go next

- This unit asked how a model behaves on data it has not seen, and which variables deserve to be in it.
- **Unit 5** asks what to do when the straight line itself is the problem: residuals that fan or bend, transformations that fix them, and decision trees, which find shapes you did not think to look for.
- Everything in this unit about honest error carries straight over. A tree has a complexity dial too, and it is chosen the same way.